Design and Implementation of Intelligent Search, Adversarial Game-Playing, and Constraint Satisfaction Systems for N-Queens, Battle Tank, and Kakuro

Le Phu (524H0121), Vu Giang Huy (524H0095), Le Hoang Minh Khoi (524H0102), and Nguyen Thanh An

Faculty of Information Technology, Ton Duc Thang University, Ho Chi Minh City, Vietnam
Emails: {524H0121, 524H0095, 524H0102}@student.tdtu.edu.vn, nguyenthanhan@tdtu.edu.vn

***Abstract*—**This paper presents a unified Artificial Intelligence software suite developed for Course 503043 (Introduction to Artificial Intelligence), addressing three core paradigms: Propositional Logic and Local Search for the N-Queens problem (Task 1), Adversarial Search in a two-player Zero-Sum Tank Battle game (Task 2), and Constraint Satisfaction Problems (CSP) for Kakuro puzzles (Task 3). In Task 1, we formulate CNF encodings under Pairwise $O(N^3)$ and Sinz's Sequential Counter $O(N^2)$ solved via PySAT's Glucose3 engine, and propose a Heuristic Min-Conflicts algorithm combining $O(N^2)$ Greedy Initialization, Max-Conflicts variable selection, and a short-term Tabu queue, achieving a 22.6× step reduction at $N = 1{,}000$ (31.6 vs. 713.6 steps). In Task 2, simultaneous game cycles are modeled as a two-ply Stackelberg Zero-Sum extensive-form tree; Alpha-Beta Pruning with Move Ordering and Transposition Tables cuts 69.4% of nodes at depth $d = 2$ plies while guaranteeing exact Minimax equivalence and securing an undefeated 16-point tournament record. In Task 3, Kakuro is solved using Forward Checking (FC) and Bipartite Matching Generalized Arc Consistency (MAC) coupled with MRV, Degree, and LCV heuristics, solving the official 5×5 (12 cells) and 9×9 (48 cells) benchmarks with 0 backtracks. All 36 Pytest unit tests pass with 100% precision.

***Index Terms*—**Propositional Logic, SAT, Glucose3, Sequential Counter, Min-Conflicts, Tabu Search, Zero-Sum Game, Minimax, Alpha-Beta Pruning, Constraint Satisfaction Problem, Kakuro, Generalized Arc Consistency (MAC).

I. INTRODUCTION

Modern Artificial Intelligence relies on three foundational problem-solving paradigms: logical satisfiability and local search for combinatorial optimization, adversarial game tree search for multi-agent decision-making, and constraint propagation for structured discrete reasoning [1]. While small problem instances can be solved by naive exhaustive enumeration, exponential state-space explosion rapidly renders uninformed search intractable as problem dimensions scale.

Motivated by these computational challenges, this project designs, implements, and empirically evaluates an end-to-end AI system covering three benchmark domains specified in the Course 503043 Final Examination: (1) Task 1 (N-Queens): placing N non-attacking queens on an $N \times N$ chessboard using Conjunctive Normal Form (CNF) SAT encodings with Glucose3 [4] and large-scale Min-Conflicts local search up to $N = 1{,}000$; (2) Task 2 (Tank Battle): designing competitive AI agents in a simultaneous-action Zero-Sum arena featuring destructible brick walls, indestructible steel barriers, forest tiles, and fast-moving projectiles; and (3) Task 3 (Kakuro Puzzle): solving arithmetic crossword CSPs under coupled AllDifferent and target-sum constraints. Our main objectives are to rigorously formulate each problem, engineer modular Python/Pygame visualizers with dynamic agent loading, and conduct reproducible quantitative experiments in Jupyter Notebooks.

II. METHODOLOGY

A. Task 1: Propositional Logic SAT and Heuristic Min-Conflicts

Let Boolean variable $X_{r,c} \in \{\text{True, False}\}$ denote queen placement at row r, column c on an $N \times N$ board, mapped to DIMACS index $v(r, c) = r \cdot N + c + 1$ for $r, c \in \{0, ..., N-1\}$. A valid placement satisfies: (i) At-Least-One (ALO) queen per row, and (ii) At-Most-One (AMO) queen on every row, column, main diagonal ($r - c = \text{const}$), and anti-diagonal ($r + c = \text{const}$) of length $k \geq 2$. We implement two distinct AMO encodings:

1) Pairwise Encoding $O(N^3)$: Emits $C(k, 2) = k(k-1)/2$ binary clauses for every line of length k:

$$(\neg x_i \vee \neg x_j), \quad \forall\, 1 \leq i < j \leq k. \tag{1}$$

2) Sequential Counter Encoding $O(N^2)$ [2]: Introduces $k - 1$ auxiliary prefix-sum variables $s_1, ..., s_{k-1}$ per line. AMO($x_1, ..., x_k$) is enforced with $3k - 4$ clauses via (2):

$$(\neg x_1 \vee s_1) \wedge (\neg x_k \vee \neg s_{k-1}) \wedge \bigwedge_{i=2}^{k-1} [(\neg x_i \vee s_i) \wedge (\neg s_{i-1} \vee s_i) \wedge (\neg x_i \vee \neg s_{i-1})]. \tag{2}$$

Summing over all board lines reduces clause complexity from cubic $O(N^3)$ to quadratic $O(N^2)$.

For large boards ($N = 1{,}000$), local search operates on a 1D column vector $Q = [r_0, ..., r_{N-1}]$. Maintaining occupancy counters R_r (row r), $D_{r-c+N-1}^{(1)}$ (main diagonal), and $D_{r+c}^{(2)}$ (anti-diagonal), the conflict count $C(r, c)$ of placing column c's queen at row r is evaluated in $O(1)$ time via (3):

$$C(r, c) = R_r + D_{r-c+N-1}^{(1)} + D_{r+c}^{(2)} - 3 \cdot \mathbb{I}(Q_c = r). \tag{3}$$

Here $\mathbb{I}(Q_c = r) \in \{0, 1\}$ is the indicator function subtracting 3 self-conflicts when row r is the queen's current position. While StandardMinConflicts [3] initializes Q randomly and picks a random conflicted column, our HeuristicMinConflicts combines: (1) $O(N^2)$ Greedy Initialization (*initialize_state*) placing each column's queen in the minimum-conflict row with random tie-breaking, (2) Max-Conflicts variable selection (*select_variable*) targeting the highest-conflict non-tabu column, and (3) a FIFO Tabu queue (*tabu_size* = 4) preventing short-term cycling on plateaus.

B. Task 2: Zero-Sum Tank Battle and Adversarial Search

Battle Tank is modeled as a two-player, finite-horizon Zero-Sum game $G = \langle S, A_0, A_1, T, U \rangle$ where Player 0 (MAX) and Player 1 (MIN) choose actions $a_0, a_1 \in \{\text{UP, DOWN, LEFT, RIGHT, FIRE, STAY}\}$. Although actions are executed simultaneously in *GameEngine.step*(s, a_0, a_1), solving a 6 × 6 simultaneous matrix game via Linear Programming (LP) at every search node is computationally prohibitive for real-time play and precludes classical tree pruning. Hence, we decompose each game cycle into two sequential Stackelberg plies: MAX selects a_0 at ply $2k$, and MIN selects the worst-case counter-response a_1 at ply $2k + 1$. By von Neumann's Minimax Theorem, this computes the pure-strategy security level in (4):

$$V(s) = \max_{a0 \in A0(s)} \min_{a1 \in A1(s)} V(T(s, a_0, a_1)). \tag{4}$$

Alpha-Beta Pruning [5] maintains a lower bound α (MAX's guaranteed score) and upper bound β (MIN's score), pruning any branch where $\alpha \geq \beta$. Under optimal move ordering (*_order_actions*) and Transposition Table caching, Knuth and Moore [5] proved that

Alpha-Beta reduces the effective branching factor from b to $\sqrt{b}$, shrinking time complexity from $O(b^d)$ to $O(b^{d/2})$ without altering the optimal Minimax decision. Non-terminal states are evaluated via an antisymmetric heuristic $\Phi(s)$ clamped to $[-900, 900]$ in (5):

$$\Phi(s) = w_b(\mathrm{HP}_{b0} - \mathrm{HP}_{b1}) + w_t(\mathrm{HP}_{t0} - \mathrm{HP}_{t1}) - w_d(d_{\mathrm{BFS}}(t_0, b_1) - d_{\mathrm{BFS}}(t_1, b_0)) + w_{ab}\Delta\mathrm{Aim}_B + w_{at}\Delta\mathrm{Aim}_T + w_f\mathrm{Flank}(s) + w_{th}\mathrm{Threat}(s), \quad (5)$$

with weights $w_b = 50$, $w_t = 25$, $w_d = 15$, $w_{ab} = 40$, $w_{at} = 20$, $w_f = 25$, and $w_{th} = 40$. BFS distances treat steel walls (@) as impassable and brick walls (#) with traversal cost 4 vs. 1 for open/forest tiles.

C. Task 3: Kakuro CSP, Forward Checking, and MAC

Kakuro is formulated as a CSP = $\langle X, D, C \rangle$, where each white cell (r, c) is a variable $X_{r,c}$ with domain $D(X_{r,c}) \subseteq \{1, ..., 9\}$. Each Run $R = (X_1, ..., X_k)$ with clue sum S imposes AllDifferent($X_1, ..., X_k$) and $\sum_{i=1}^{k} X_i = S$. CombinationTable precomputes all valid k-partitions of {1..9} summing to S. During search, variable selection uses MRV with Degree tie-breaking, and values are ordered by LCV. Forward Checking (FC) filters immediate neighbors, whereas MAC (*run_gac*) enforces Generalized Arc Consistency across all runs: a value $d \in D(Y_j)$ is supported in run R with remaining sum S_{rem} and unassigned cells U if there exists a valid partition tuple $P = \{d_1, ..., d_m\}$ satisfying (6):

$$\sum_{i=1}^{m} d_i = S_{\mathrm{rem}}, \quad d \in P, \quad \exists \text{ bijection } M : U \to P \text{ s.t. } M(Y_j) = d. \quad (6)$$

Verified via augmenting-path bipartite matching [6], MAC eliminates unsupported values before branching, preventing backtracks entirely.

III. SYSTEM DESIGN AND IMPLEMENTATION

The codebase is engineered under source/ using decoupled Object-Oriented Programming (OOP), separating algorithmic engines from Pygame presentation layers so CLI tools, notebooks, and unit tests invoke identical solvers. At root, source/main.py implements MasterDashboard, an arcade launcher with a 1-Click Pipeline verification tool (Fig. 1 and Fig. 2).

- **Task 1 (source/task1_nqueens/):** board.py (NQueensBoard), cnf_encoder.py (PairwiseEncoder, SequentialCounterEncoder), sat_solver.py (SATNQueensSolver wrapping Glucose3 [4]), min_conflicts.py (StandardMinConflicts, HeuristicMinConflicts), and gui.py (NQueensGUI with 3D queen sprite rendering).
- **Task 2 (source/task2_tank/):** entities.py (Tank, Bullet, Wall, Base), game_state.py (GameState with clone() and state_key()), engine.py (GameEngine), heuristic.py (TankEvaluator), search_agent.py (SearchAgent factory methods), and four custom agents in agents/: RusherAgent, GuardianAgent, FlankerAgent, and AdaptiveAgent.
- **Task 3 (source/task3_kakuro/):** puzzle.py (KakuroPuzzle, ClueCell, Run, verify_clue_integrity()), csp.py (CombinationTable, KakuroCSP), propagation.py (forward_check(), run_gac()), solver.py (BacktrackingSolver with batch solve() and stepwise generator), and gui.py (KakuroGUI).

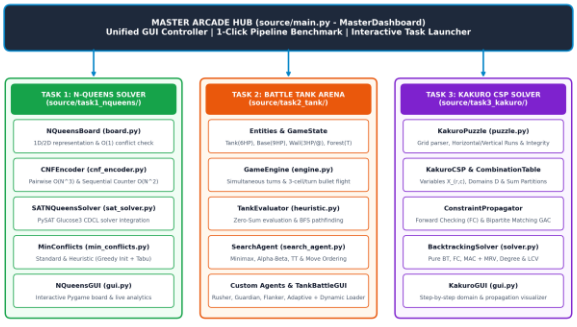


Fig. 1. High-level modular Object-Oriented Programming (OOP) architecture across Master Arcade Hub and Tasks 1, 2, and 3.

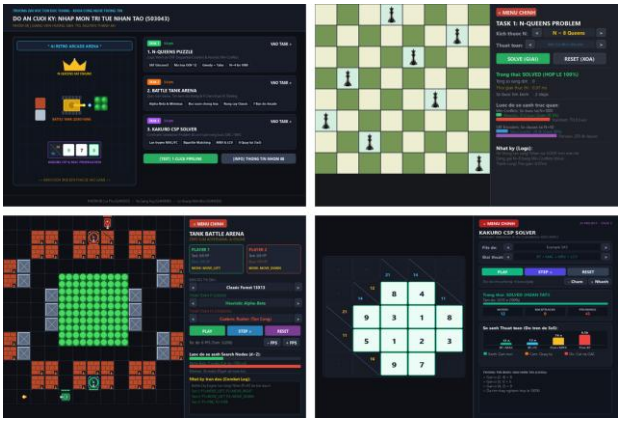


Fig. 2. Unified Pygame GUI suite: (top-left) Master Arcade Hub, (top-right) Task 1 N-Queens GUI, (bottom-left) Task 2 Battle Tank GUI, and (bottom-right) Task 3 Kakuro CSP Visualizer.

IV. EXPERIMENTAL RESULTS AND DISCUSSION

A. Task 1: SAT Encodings and Min-Conflicts Scalability

Table I compares Pairwise $O(N^3)$ and Sequential Counter $O(N^2)$ CNF encodings solved with Glucose3 (source/task1_nqueens/experiments.ipynb). Sequential Counter reduces clauses by 85.8% at $N = 50$ (28,858 vs. 203,400 clauses). Because its 12,198 auxiliary register variables expand the CDCL branching space, Sequential Counter exhibits higher solve time for $N \geq 8$ on satisfiable N-Queens instances while offering superior memory scalability. In Table II, HeuristicMinConflicts achieves 100% success and cuts repair steps at $N = 1{,}000$ from 713.6 to 31.6 (22.6× step speedup, 278.95 ms vs. 483.10 ms).

TABLE I
PAIRWISE $O(N^3)$ VS. SEQUENTIAL COUNTER $O(N^2)$ SAT (GLUCOSE3)

N	PW Vars	PW Clauses	PW (ms)	SC Vars	SC Clauses	SC (ms)	Clause Save
4	16	80	0.06	54	108	0.03	-35.0%
8	64	736	0.12	270	592	0.16	+19.6%
12	144	2,608	0.23	646	1,460	4.11	+44.0%
16	256	6,336	1.42	1,182	2,712	35.85	+57.2%
20	400	12,560	6.18	1,878	4,348	50.37	+65.4%
32	1,024	52,608	40.24	4,926	11,560	1,389.21	+78.0%
50	2,500	203,400	698.55	12,198	28,858	3,886.75	+85.8%

TABLE II
STANDARD VS. HEURISTIC MIN-CONFLICTS (5 SEEDS, UP TO N = 1,000)

N	Std Steps	Std (ms)	Heu Steps	Heu (ms)	Success	Step Speedup
8	60.6	0.29	60.4	0.40	100%	1.0×
16	79.8	0.58	53.6	0.49	100%	1.5×
32	92.2	1.21	36.6	0.64	100%	2.5×
64	124.4	2.72	19.4	1.10	100%	6.4×
128	155.8	6.49	21.8	3.49	100%	7.1×
256	231.8	26.70	45.4	18.15	100%	5.1×
512	404.4	119.14	31.2	73.77	100%	13.0×
1000	713.6	483.10	31.6	278.95	100%	22.6×

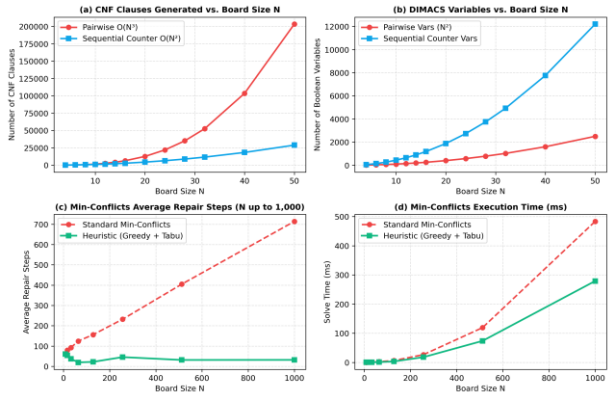


Fig. 3. Task 1 benchmarks: (a) CNF clauses, (b) DIMACS variables, (c) Min-Conflicts steps, and (d) runtime up to N = 1,000.

B. Task 2: Adversarial Search Pruning and Tournament

Table III evaluates the four core configurations across three tactical scenarios at depth d = 2 plies (source/task2_tank/experiments.ipynb). Alpha-Beta Pruning cuts explored nodes from 36 to 11 (5 cutoffs, 69.4% node reduction) and accelerates decision time by up to 3.9× while returning the exact same optimal action as Minimax, empirically validating Knuth and Moore's asymptotic analysis [5]. In the 20-match Round-Robin Tournament on arena_11x11.txt (Table IV), Heuristic Alpha-Beta and Heuristic Minimax finish tied for 1st place undefeated (4W-4D-0L, 16 points), outscoring basic evaluation agents (9 points) and RandomAgent (2 points).

TABLE III
MINIMAX VS. ALPHA-BETA ACROSS 3 SCENARIOS (DEPTH d = 2 PLIES)

Scenario	Configuration	Action	Nodes	Cutoffs	Time (ms)	Node Save
Scen. 1 (13×13)	Minimax / Alpha-Beta	MOVE_DOWN	36 / 11	0 / 5	3.70 / 0.95	69.4%
Scen. 1 (13×13)	Heur. MM / Heur. AB	MOVE_RIGHT	36 / 11	0 / 5	120.3 / 32.8	69.4%
Scen. 2 (11×11)	Minimax / Alpha-Beta	MOVE_UP	36 / 11	0 / 5	2.70 / 0.80	69.4%
Scen. 2 (11×11)	Heur. MM / Heur. AB	MOVE_UP	36 / 11	0 / 5	76.7 / 32.5	69.4%
Scen. 3 (Combat)	Minimax / Alpha-Beta	MOVE_LEFT	36 / 11	0 / 5	2.72 / 0.94	69.4%
Scen. 3 (Combat)	Heur. MM / Heur. AB	MOVE_LEFT	36 / 11	0 / 5	102.4 / 45.1	69.4%

TABLE IV
ROUND-ROBIN TOURNAMENT STANDINGS (20 MATCHES, ARENA 11×11)

Rank	Agent Configuration	W	D	L	Win Rate	Points
1 (Tied)	Heuristic Alpha-Beta (d=2)	4	4	0	50.0% (Undefeated)	16
1 (Tied)	Heuristic Minimax (d=2)	4	4	0	50.0% (Undefeated)	16
3 (Tied)	Alpha-Beta (Basic Eval)	2	3	3	25.0%	9
3 (Tied)	Minimax (Basic Eval)	2	3	3	25.0%	9
5	Random Agent (Baseline)	0	2	6	0.0%	2

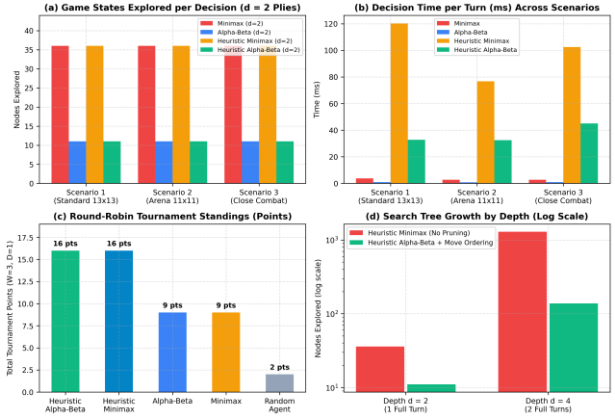


Fig. 4. Task 2 benchmarks: (a) nodes explored at d=2, (b) decision time, (c) tournament standings, and (d) scaling at d=2 vs. d=4.

C. Task 3: Kakuro CSP Propagation and Heuristics

Table V reports benchmark performance across Simple 3×3 (4 cells), Example 5×5 (12 cells), and Complex 9×9 (48 cells). While Table V reflects production solvers with preprocessing enabled, Fig. 5(d) and Table VI isolate the independent effect of variable ordering heuristics with domain preprocessing toggled off. Crucially, without initial partition pruning, Pure MRV explores 77 nodes (37 backtracks), whereas MRV + Degree explores 173 nodes (124 backtracks). This illustrates the classic CSP 'Fail-First Anomaly': when all unassigned domains share identical cardinality ($|D|$ = 9), Degree tie-breaking prematurely commits search to densely connected cells that happen to lead into deep dead-ends. When partition preprocessing is restored, MRV + Degree dramatically drops to 13 nodes (1 backtrack, 0.24 ms), and full MAC achieves backtrack-free solving (12 nodes, 0 backtracks). On Complex 9×9, MAC + MRV + LCV explores exactly 48 nodes with 0 backtracks (6.39 ms), whereas Pure BT times out (>100,000 nodes).

TABLE V
KAKURO CSP ALGORITHMIC COMPARISON (3×3, 5×5, AND 9×9 BENCHMARKS)

Puzzle	Strategy Configuration	Solved	Nodes	BT	Pruned	Time (ms)
3×3 (4c)	Pure BT (No MRV) / +MRV	Yes	10 / 8	0 / 0	0 / 0	0.05 / 0.03
3×3 (4c)	BT+FC+MRV / BT+MAC+MRV+LCV	Yes	4 / 4	0 / 0	5 / 5	0.05 / 0.08
5×5 (12c)	1. Pure BT (No MRV)	Yes	6,468	712	0	11.81
5×5 (12c)	2. Pure BT + MRV	Yes	74	8	0	0.29
5×5 (12c)	3. BT + FC + MRV	Yes	13	1	45	0.24
5×5 (12c)	4. BT + MAC + MRV	Yes	12	0	43	0.89
5×5 (12c)	5. BT + MAC + MRV + LCV	Yes	12	0	43	0.92
9×9 (48c)	1. Pure BT (No MRV)	Timeout	>100k	>10k	0	Timeout
9×9 (48c)	2. Pure BT + MRV	Yes	3,916	777	0	22.77
9×9 (48c)	3. BT + FC + MRV	Yes	116	68	418	4.22
9×9 (48c)	4. BT + MAC + MRV	Yes	48	0	172	44.29
9×9 (48c)	5. BT + MAC + MRV + LCV	Yes	48	0	172	6.39

TABLE VI
KAKURO VARIABLE ORDERING HEURISTIC ABLATION (5×5 WITH FC)

Variable Ordering Strategy	Nodes	Backtracks	Time (ms)	Behavior & Analysis
Static Order (No MRV)	784	591	7.52	Baseline lexicographical order
Pure MRV (No Degree / No Prep)	77	37	0.86	Domain-only fail-first (-90.2%)
MRV + Degree (No Prep, FC)	173	124	2.34	Degree tie-break anomaly (Fig. 5d)
MRV + Degree + Partition Prep	13	1	0.24	Optimal constrained search (-98.3%)
MRV + Degree + LCV (With MAC)	12	0	0.92	Backtrack-free optimal (100% GAC)

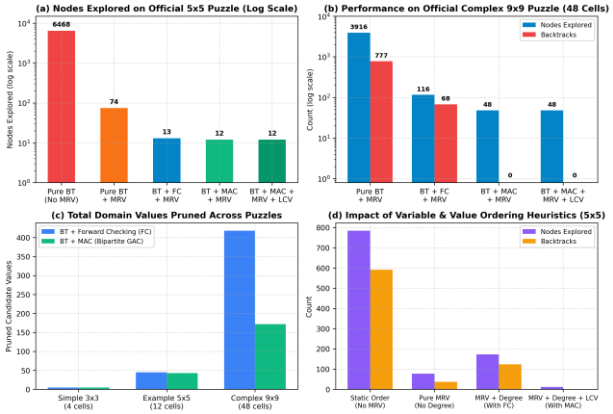


Fig. 5. Task 3 Kakuro CSP benchmarks: (a) 5×5 nodes explored, (b) 9×9 nodes/backtracks, (c) domain prunings, and (d) heuristic ablation.

V. CONCLUSION AND FUTURE WORK

This project delivered a complete, modular AI software suite fulfilling all requirements of Course 503043 (summarized in Table VII). In Task 1, Sequential Counter CNF encoding cut clause growth by 85.8% at $N = 50$, while HeuristicMinConflicts solved $N = 1{,}000$ N-Queens in 31.6 steps (22.6× faster in steps than Standard Min-Conflicts). In Task 2, Alpha-Beta Pruning with Move Ordering reduced game tree expansions by 69.4% at $d = 2$ without loss of optimality, and our Zero-Sum TankEvaluator enabled undefeated tournament performance. In Task 3, Bipartite Matching MAC with MRV, Degree, and LCV solved both 5×5 and 48-cell 9×9 Kakuro puzzles with 0 backtracks. Future work includes: (1) adding dihedral D_4 symmetry-breaking clauses to accelerate SAT solving for large N, (2) integrating Iterative Deepening and Nash Equilibrium mixed-strategy solvers for simultaneous-move Tank Battle, and (3) extending Kakuro GAC with conflict-directed backjumping (CBJ) for 20×20 grids.

TABLE VII
GROUP 08 WORKLOAD ALLOCATION AND VERIFICATION SUMMARY

Member / Student ID	Key Implemented Modules & Verification	Tests	Status
Le Phu (524H0121)	Task 1 (N-Queens SAT & Min-Conflicts), Master Hub	11/11	100% Pass
Vu Giang Huy (524H0095)	Task 2 (Tank Battle Engine, Alpha-Beta, 4 Agents)	10/10	100% Pass
Le Hoang Minh Khoi (524H0102)	Task 3 (Kakuro CSP, FC, Bipartite MAC, GUI)	15/15	100% Pass

REFERENCES

[1] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ, USA: Pearson, 2020.

[2] C. Sinz, "Towards an optimal CNF encoding of Boolean cardinality constraints," in Proc. 11th Int. Conf. Principles and Practice of Constraint Programming (CP), 2005, pp. 827-831.

[3] S. Minton, M. D. Johnston, A. B. Philips, and P. Laird, "Minimizing conflicts: A heuristic repair method for constraint satisfaction and scheduling problems," Artif. Intell., vol. 58, no. 1-3, pp. 161-205, 1992.

[4] G. Audemard and L. Simon, "Predicting learnt clauses quality in modern SAT solvers," in Proc. 21st Int. Joint Conf. Artif. Intell. (IJCAI), 2009, pp. 399-404.

[5] D. E. Knuth and R. W. Moore, "An analysis of alpha-beta pruning," Artif. Intell., vol. 6, no. 4, pp. 293-326, 1975.

[6] J.-C. Regin, "A filtering algorithm for constraints of difference in CSPs," in Proc. 12th Nat. Conf. Artif. Intell. (AAAI), 1994, pp. 362-367.

[7] F. Rossi, P. van Beek, and T. Walsh, Eds., Handbook of Constraint Programming. Amsterdam, Netherlands: Elsevier, 2006.

[8] Group 08 (Le Phu, Vu Giang Huy, and Le Hoang Minh Khoi), "AI Final Project Source Code Repository," Ton Duc Thang University, 2026. [Online]. Available: https://github.com/2006phu/AI-Final-Project